Department of
Computer Science and Engineering

Title: Alpha Beta Pruning

ARTIFICIAL INTELLIGENCE LAB

CSE 316/404

Green University of Bangladesh

1 Objective(s)

- To understand the concept of Alpha-Beta Pruning in Artificial Intelligence.
- To implement Alpha-Beta Pruning in a Minimax search tree to optimize decision-making in two-player games.

2 Problem analysis

Alpha-Beta Pruning is an optimization technique applied to the Minimax algorithm, which is widely used in game-playing AI for decision-making. It helps to reduce the number of nodes evaluated in the search tree without affecting the final decision, thus making the algorithm more efficient.

1. In the **Minimax** algorithm, two players; generally referred to as MAX and MIN, take turns making moves.
2. The **MAX** player tries to maximize the score (best possible outcome for itself).
3. The **MIN** player tries to minimize the score (worst possible outcome for MAX).

However, exploring all possible game states is computationally expensive, especially for games with large search trees such as Chess or Tic-Tac-Toe. Alpha-Beta Pruning helps to cut off branches that will not influence the final decision, thus avoiding unnecessary evaluations.

2.1 Alpha-Beta Pruning in Minimax Search Tree

The Alpha-Beta Pruning process introduces two parameters:

1. Alpha (α): The best value that the MAX player can guarantee at that point or above.
2. Beta (β): The best value that the MIN player can guarantee at that point or below.

During the traversal of the game tree:

1. If the current node's value becomes worse than the previously explored option, the branch is pruned (i.e., not further explored).
2. The pruning condition occurs when $\beta \leq \alpha$, meaning further exploration of that branch will not yield a better outcome for either player.

This pruning does not affect the correctness of the Minimax result. It only skips evaluating parts of the tree that cannot influence the final decision.

2.2 Illustration of Alpha-Beta Pruning

In a game tree:

1. The MAX node selects the maximum value among its child nodes.
2. The MIN node selects the minimum value among its child nodes.
3. As the algorithm explores the tree depth-first, α and β values are updated based on the best options found so far.
4. When a node's value is outside the $\alpha - \beta$ window, the remaining siblings of that node are skipped (pruned).

In the Following example in Figure 1,

(a) First, we will take care of the first move. So initially, we will define the worst case $\alpha = -\infty$ and $\beta = +\infty$. If alpha is greater than or equal to beta, we will prune the node.

(b) We didn't prune it since the initial value of alpha is less than beta. Now it's turn for MAX. Therefore, we will calculate the value of alpha at node D. At node D, the value of alpha will be max $(2, 3) = 3$. Now, node B's turn is MIN. That means that the value of alpha beta at node B will be min $(3, \infty)$. Therefore, alpha will be $\breve{}\infty$ and beta will be 3 at node B. Next, algorithms will pass the values of $\alpha = -\infty$ and $\beta = 3$ to the next successor of Node B, that is node E.

© Dept. of Computer Science and Engineering, GUB

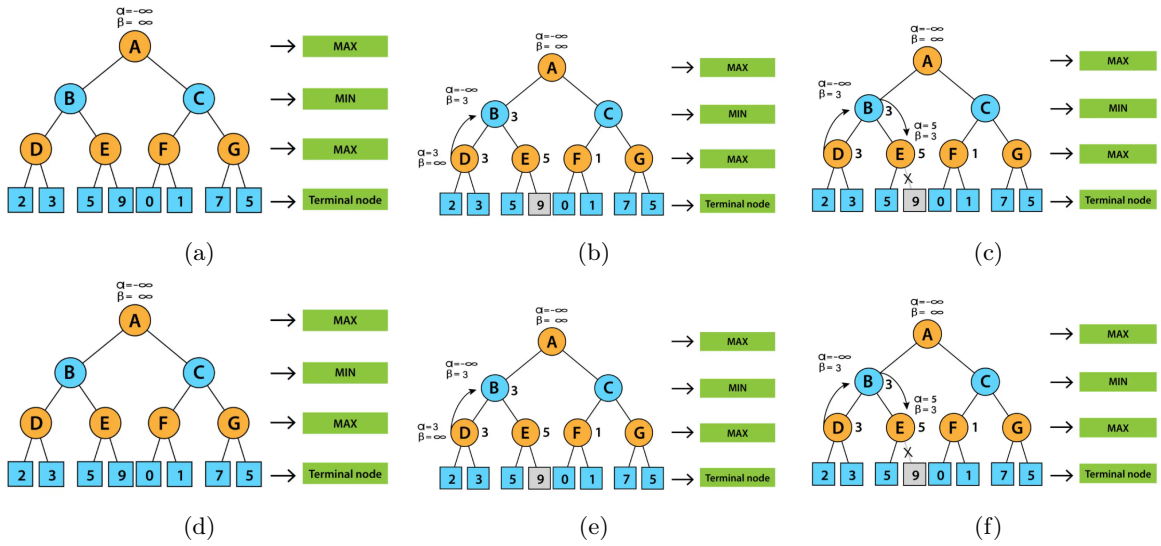

Figure 1: Alpha-Beta Pruining Example

(c) Now it's turn for MAX. Therefore, we will search for MAX at node E. The value of alpha at E is $\breve{}\infty$ and will be compared with 5. So, MAX $(-\infty, 5)$ will be 5. Thus, at node E, alpha = 5, Beta = 5. We now know that alpha is greater than beta and is also a pruning condition, so we can prune the right successor of node E, and the algorithm will not be visited, and the value of node E will be 5.

(d) In the next step, the algorithm again comes from node B to node A. The alpha of node A is changed to the max value of MAX$(-\infty, 3)$. Now, alpha and beta at node A are $(3, +\infty)$ and will be transmitted to node C. The same values will be transferred to node F. The value of alpha will be compared to the left branch, which is 0 at node F. Now, MAX $(0, 3)$ will be 3 and is compared with the right child (1) and MAX(3, 1) = 3 still α is 3, but the node value of F will be 1.

(e) Afterward, node F will pass the node value 1 to C and compare it to the beta value at C. Now it's turn for MIN. So, MIN $(+\infty, 1)$ will be 1. Now, at node C, $\alpha = 3, \beta = 1$, and alpha is greater than beta, which again satisfies the pruning condition. Then, the successor of node C, G, will be pruned, and the algorithm did not calculate the whole subtree G.

(f) Now, C will return its node value to A, and A will calculate MAX(1, 3), resulting in 3.

3 Implementation in Python

```
import math

def minimax (curDepth, nodeIndex, maxTurn, scores, targetDepth):
  #base case : targetDepth reached
 if (curDepth == targetDepth):
  return scores[nodeIndex]

 if (maxTurn):
  return max(minimax(curDepth + 1, nodeIndex * 2, False, scores,targetDepth),
      minimax(curDepth + 1, nodeIndex * 2 + 1, False, scores, targetDepth))

 else:
  return min(minimax(curDepth + 1, nodeIndex * 2, True, scores, targetDepth),
      minimax(curDepth + 1, nodeIndex * 2 + 1, True, scores, targetDepth))

# Driver code
```

```
scores = []
n = int(input())
for i in range(n):
  x = int(input())
  scores.append(x)

treeDepth = math.log(len(scores), 2)

print("The optimal value is : ", end = "")
print(minimax(0, 0, True, scores, treeDepth))
```

4 Input/Output

4.1 Input

Input of the program is given below.

```
Enter the elements: 8
scores = 3, 5, 2, 9, 12, 5, 23, 23
```

4.2 Output

Output of the program is given below.

```
The optimal value is : 12
```

5 Discussion & Conclusion

Based on the focused objective(s) to understand about alpha beta pruning algorithm, the additional lab exercise made more confident towards the fulfillment of the objectives(s).

6 Lab Task (Please implement yourself and show the output to the instructor)

1. Visualization and Analysis of Alpha-Beta Pruning.

6.1 Problem analysis

Write a Python program that:

1. Generates a random binary tree of leaf scores (for example, 8 or 16 leaf nodes).
2. Runs both Minimax and Alpha-Beta Pruning on the same tree.
3. Counts and displays how many nodes were visited and pruned in each case.

Finally, display the optimal value found by both algorithms, the total nodes evaluated and nodes pruned in Alpha-Beta and the percentage improvement in efficiency.

6.2 Input

Input of the program is given below.

```
No input Needed.
```

6.3 Output

Output of the program is given below.

```
Generated leaf node values: [3, 5, 2, 9, 12, 5, 23, 23]

Running Minimax...
Nodes evaluated (Minimax): 15
Optimal value: 12

Running Alpha-Beta Pruning...
Nodes evaluated (Alpha-Beta): 9
Nodes pruned: 6
Efficiency improvement: 40.0%
```

7 Lab Exercise (Submit as a report)

Implement a simple Tic-Tac-Toe game between a human player and an AI agent. The AI should use the Minimax algorithm with Alpha-Beta Pruning to choose the best move. Print how many nodes were pruned during each AI turn.

7.1 Input

Input of the program is given below.

```
1,0,1
0,1,x
x,x,0
```

7.2 Output

Output of the program is given below.

```
Nodes evaluated: 47
Nodes pruned: 22
```

8 Policy

Copying from internet, classmate, seniors, or from any other source is strongly prohibited. 100% marks will be *deducted* if any such copying is detected.

© Dept. of Computer Science and Engineering, GUB